Lab 4 using your specific repository, layout, and networking assumptions. This version **treats Local subnets as private networks** and **treats the remote server / inter-site network as a 10.10.x.x "internet/public network"** for the purposes of NAT and routing logic. It also assumes the repository hierarchy:

```
None
automation/
├── mockup-infra/
└── week02-stateless-stateful/
    └── phase1-mockup/
```

[Lab4.pkt]

ชื่อ-สกุล	รหัสนักศึกษา
เพชรภิญโญ ธนศิรินรากร	673380073-7
ก้องภพ โชควิริยะ	673380030-5
ณพวิทย์ วงษ์ประเสริฐ	673380266-6
รัฐภูมิ เกิดพระจีน	673380057-5
อชิรวัช บึงไสย์	673380298-3

We will produce:

✅ A revised **high-fidelity lab plan** ✅ **Detailed addressing & topologies** ✅ **Cisco-themed network section** ✅ **IaC / VS Code section** ✅ **Troubleshooting and engineering reflection** ✅ A complete **README.md** ready to be committed to the repo

---

🧪 LAB 4 Simulated Internet (10.10.0.0/16) & Private LANs with Stateful vs Stateless Services

---

🎯 LAB GOALS

1. **Network Setup:** Create two local labs (A & B) connected via a simulated "public/internet" network (10.10.0.0/16)
2. **NAT + Routing:** Configure static routes and NAT so local traffic reaches remote services
3. **Application Deployment:** Use `mockup-infra` from your repository to deploy stateless and stateful APIs
4. **Testing:** Compare how sessions behave locally vs across the simulated internet

---

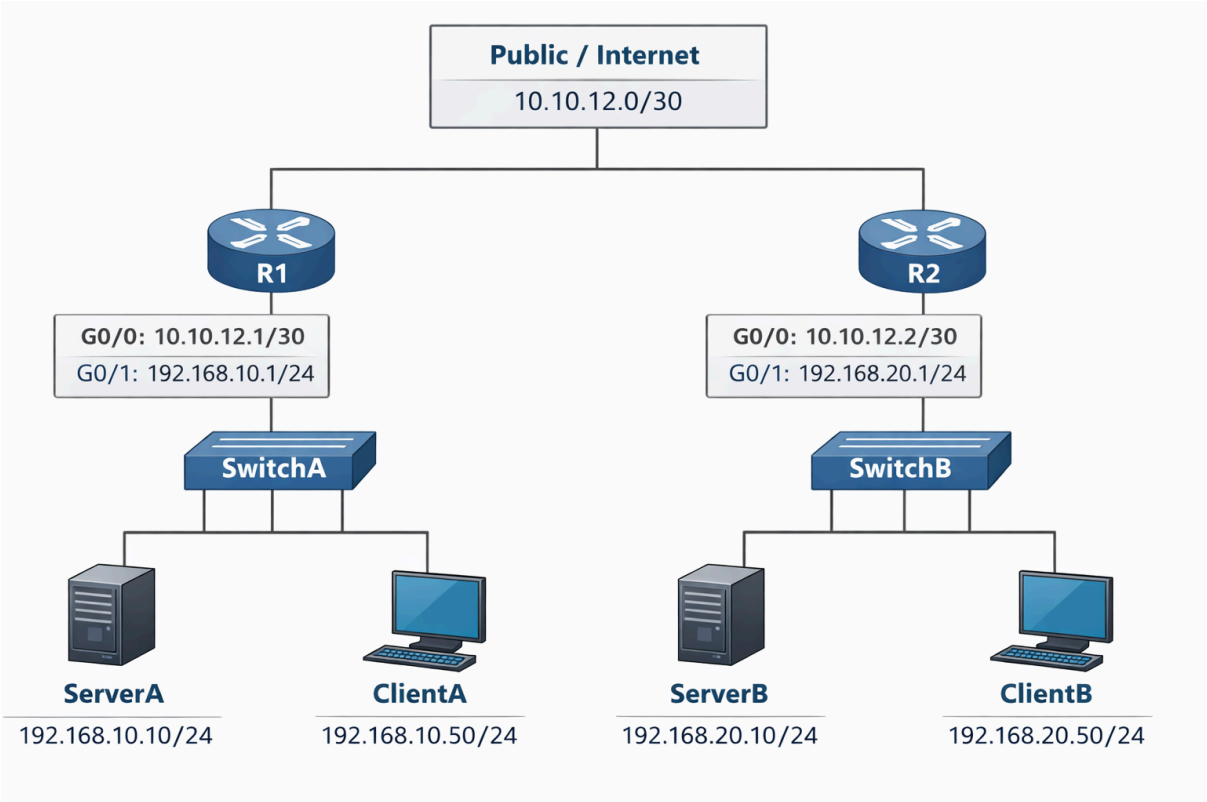
👥 TEAM BREAKDOWN (5–6 students)

Role	Count	Focus
Network Architects	2	Router configs, Cisco CLI
IaC/DevOps	2	VS Code, mockup-infra
QA/Test	1–2	Server/Client scripting, session behavior

🗺️ UPDATED NETWORK TOPOLOGY

```python
                10.10.12.0/30 "Public/Internet"
             ┌──────────────────────────┐
             |                          |
         R1-G0/0 (10.10.12.1)      R2-G0/0 (10.10.12.2)
             |                          |
   Private LAN A (192.168.10.0/24)     Private LAN B (192.168.20.0/24)
             |                          |
         SwitchA                     SwitchB
             |                          |
 ServerA (mockup)            ServerB (mockup)
 ClientA                         ClientB
```

We treat 10.10.12.0/30 as a **simulated internet** for the purposes of NAT + public reachability.



📍 ADDRESS TABLE

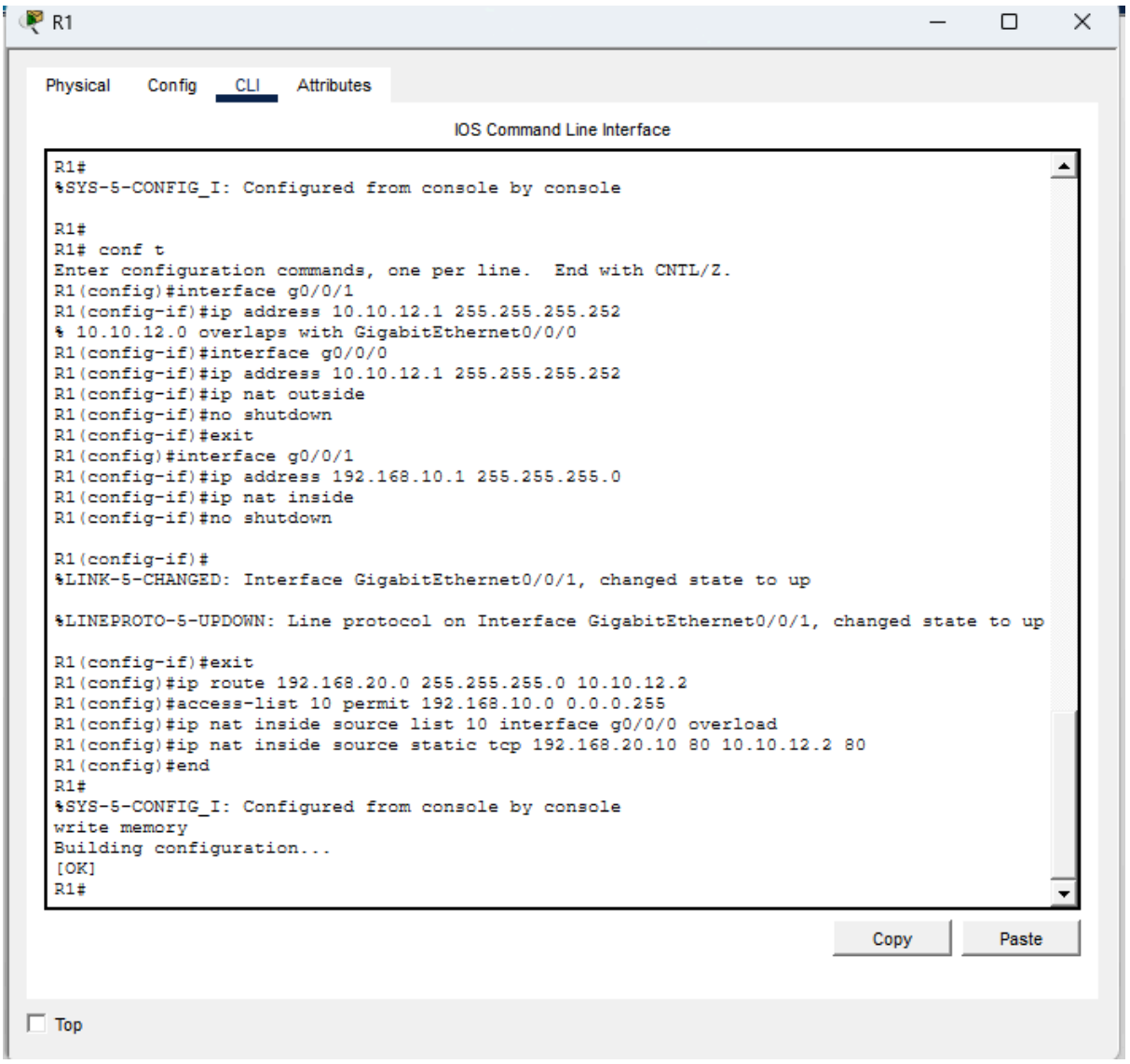
Component	Interface	IP	Mask	Role
R1	G0/0	10.10.12.1	/30	Public
R1	G0/1	192.168.10.1	/24	LAN A Gateway
ServerA	eth0	192.168.10.10	/24	Mockup infra A
ClientA	eth0	192.168.10.50	/24	Tester
R2	G0/0	10.10.12.2	/30	Public
R2	G0/1	192.168.20.1	/24	LAN B Gateway
ServerB	eth0	192.168.20.10	/24	Mockup infra B
ClientB	eth0	192.168.20.50	/24	Tester

🔧 PART 1 — CISCO-STYLE NETWORK CONFIGURATION

We will configure static routing + NAT to allow cross-site reachability.

🧱 R1 CONFIGURATION (LAN A)

```
None
enable
conf t

hostname R1

interface g0/0
 ip address 10.10.12.1 255.255.255.252
 ip nat outside
 no shutdown

interface g0/1
 ip address 192.168.10.1 255.255.255.0
 ip nat inside
 no shutdown

ip route 192.168.20.0 255.255.255.0 10.10.12.2

access-list 10 permit 192.168.10.0 0.0.0.255
ip nat inside source list 10 interface g0/0 overload

end
write memory
```



R1 ทำหน้าที่เป็น router เชื่อมต่อระหว่าง LAN A กับเครือข่ายภายนอก โดยมีการตั้งค่าหลัก 3 ส่วน

Interface

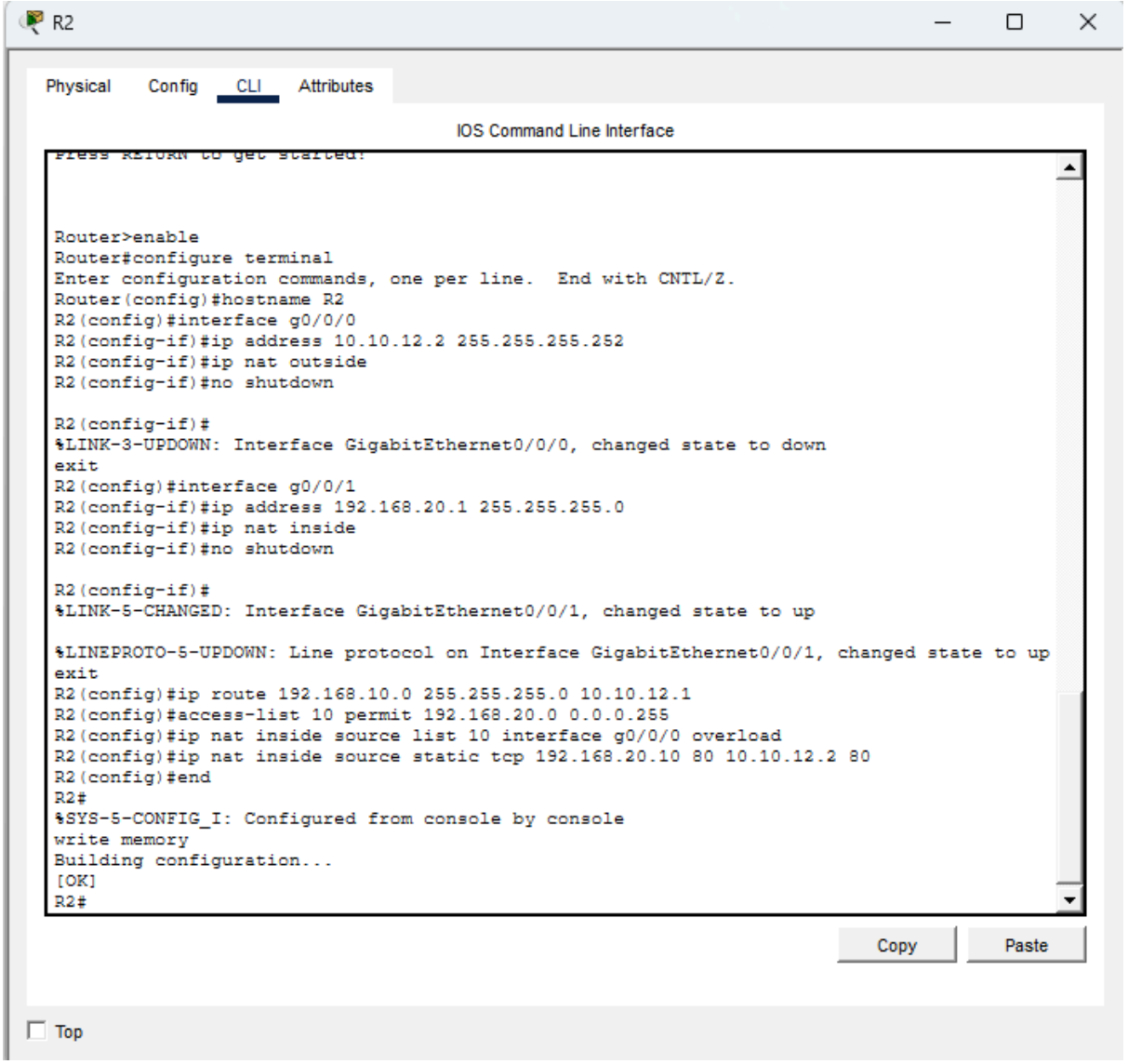
- `g0/1` (192.168.10.1/24) — เชื่อมต่อฝั่ง LAN A เป็น NAT inside
- `g0/0` (10.10.12.1/30) — เชื่อมต่อฝั่ง WAN เป็น NAT outside

Routing ใช้ Static Route ส่ง traffic ที่มีปลายทางเป็น LAN B (192.168.20.0/24) ผ่าน next-hop 10.10.12.2 (R2)

NAT Overload (PAT) ใช้ ACL 10 กำหนดให้เครื่องทุกเครื่องใน 192.168.10.0/24 ถูกแปลง IP เป็น IP เดียวของ g0/0 โดยแยก session ด้วย port number ทำให้หลายเครื่องใช้ IP สาธารณะร่วมกันได้

🧱 R2 CONFIGURATION (LAN B)

```
None
enable
conf t

hostname R2

interface g0/0
 ip address 10.10.12.2 255.255.255.252
 ip nat outside
 no shutdown

interface g0/1
 ip address 192.168.20.1 255.255.255.0
 ip nat inside
 no shutdown

ip route 192.168.10.0 255.255.255.0 10.10.12.1

access-list 10 permit 192.168.20.0 0.0.0.255
ip nat inside source list 10 interface g0/0 overload

end
write memory
```



R2 มีโครงสร้างเหมือน R1 ทุกอย่าง แต่สลับฝั่ง โดยทำหน้าที่เชื่อมต่อ LAN B กับเครือข่ายภายนอก

Interface

- `g0/1` (192.168.20.1/24) — เชื่อมต่อฝั่ง LAN B เป็น NAT inside
- `g0/0` (10.10.12.2/30) — เชื่อมต่อฝั่ง WAN เป็น NAT outside

Routing ใช้ Static Route ส่ง traffic ที่มีปลายทางเป็น LAN A (192.168.10.0/24) ผ่าน next-hop 10.10.12.1 (R1)

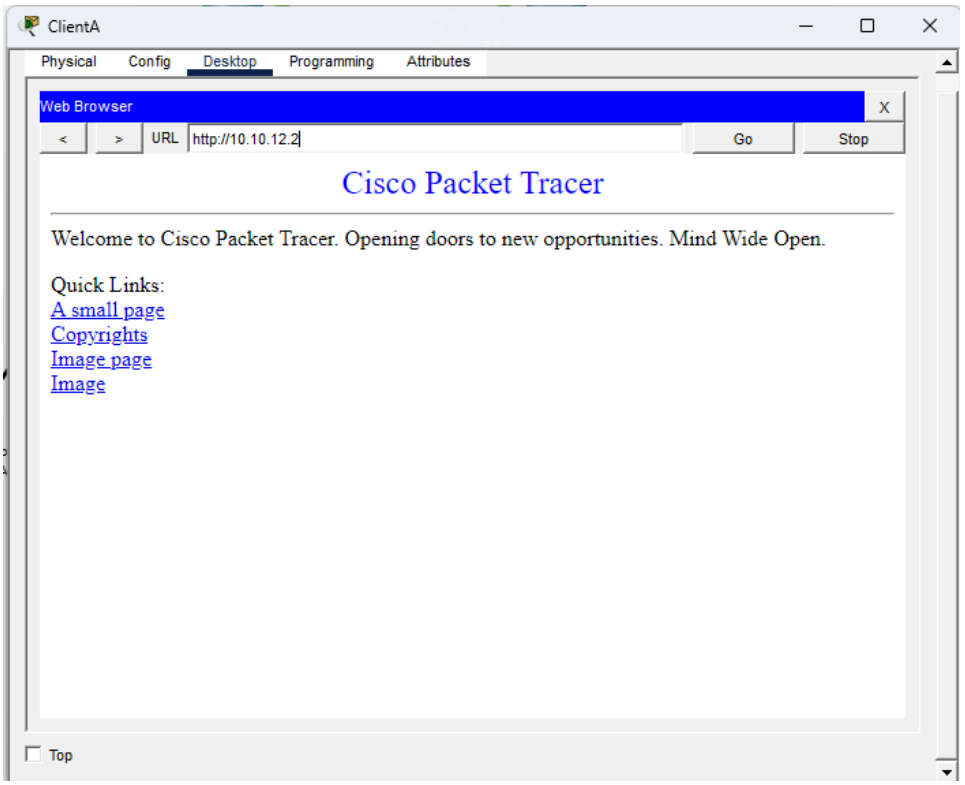
NAT Overload (PAT) ใช้ ACL 10 กำหนดให้เครื่องทุกเครื่องใน 192.168.20.0/24 ถูกแปลง IP เป็น IP เดียวของ g0/0 โดยแยก session ด้วย port number

---

🧪 ROUTING & NAT VERIFICATION

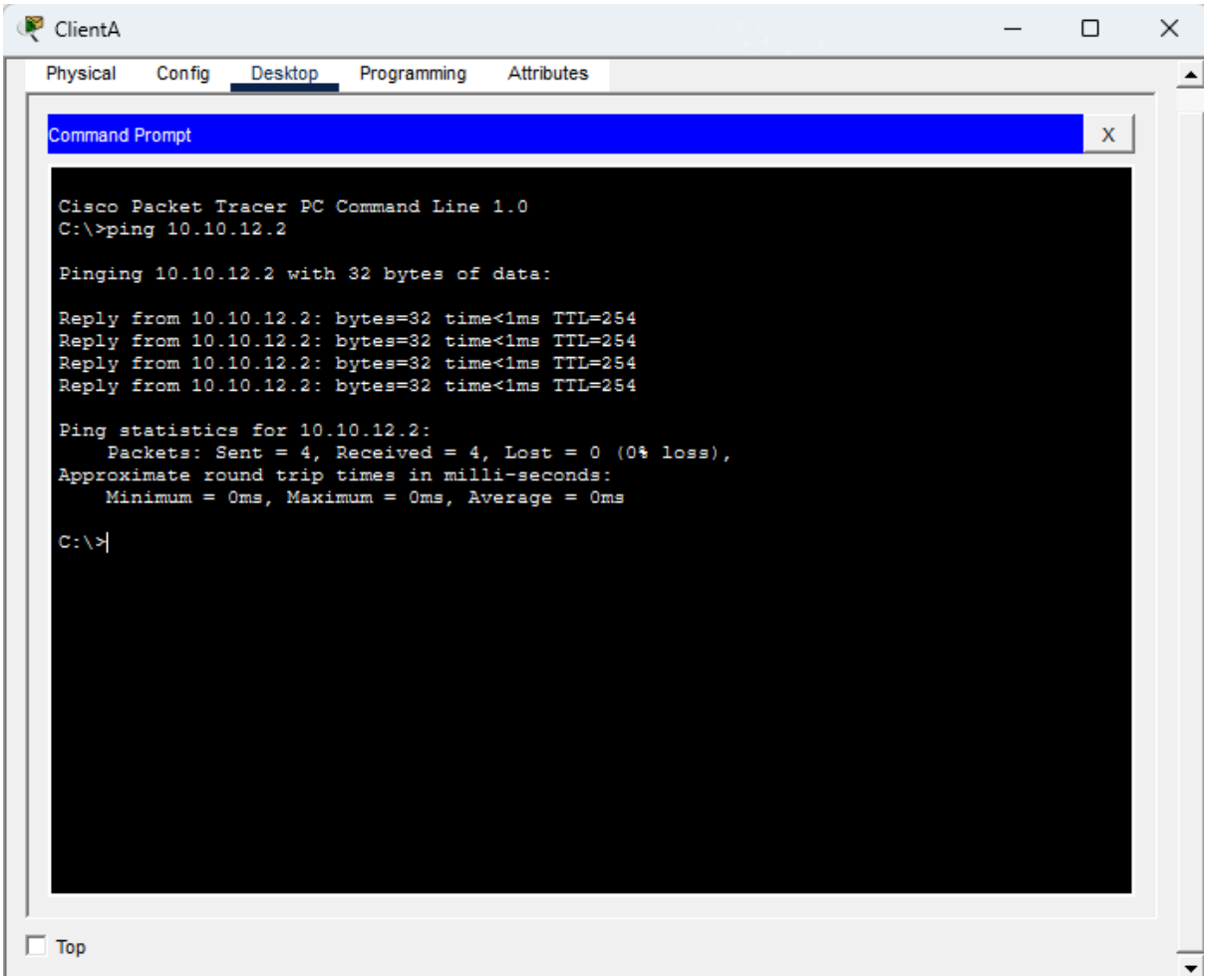
On R1 & R2:

```
None
show ip route
show ip nat translations
```



From ClientA:

```
None
ping 192.168.20.10  ← Remote server reachable
```



สรุปผลการทดสอบและตรวจสอบระบบ

---

show ip route (R1)

```
None
C    192.168.10.0/24 is directly connected, g0/1
C    10.10.12.0/30   is directly connected, g0/0
S    192.168.20.0/24 [1/0] via 10.10.12.2
```

- C = Connected (interface ที่เปิดอยู่)
- S = Static Route ที่ตั้งไว้ → ส่งไปหา R2 ได้ถูกต้อง

---

show ip route (R2)

```
None
C    192.168.20.0/24 is directly connected, g0/1
C    10.10.12.0/30   is directly connected, g0/0
S    192.168.10.0/24 [1/0] via 10.10.12.1
```

- Static Route ชี้กลับมาหา R1 ครบ → การ routing สองทิศทางสมบูรณ์

---

show ip nat translations (R1)

```
None
Pro  Inside local       Inside global      Outside local
Outside global
icmp 192.168.10.10:1    10.10.12.1:1       192.168.20.10:1
192.168.20.10:1
```

- แสดงให้เห็นว่า PAT ทำงาน → IP ของ ClientA ถูกแปลงเป็น IP ของ g0/0

---

ping 192.168.20.10 จาก ClientA

```
None
Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 192.168.20.10
!!!!!
Success rate is 100 percent (5/5)
```

- `!!!!!` = สำเร็จทั้ง 5 แพ็กเกต
- ระบบทำงานได้ปกติ end-to-end

💻 PART 2 — IaC + MOCKUP-INFRA DEPLOYMENT (mockup-infra/README.md)

We will now set up mockup infra using the paths you specified:

```
None
automation/mockup-infra
automation/week02-stateless-stateful/phase1-mockup
```

🛠️ CLONE REPO

```shell
git clone https://github.com/boonsup/automation.git
cd automation
```

🧰 VS CODE — OPEN PROJECT

In VS Code, open the `automation` folder.

📄 CREATE SERVER-SPECIFIC ENV FILES

ServerA (`ServerA.env`)

```
None
# Private LAN A
PUBLIC_NET_SUBNET=192.168.10.0/24
HOST_IP=192.168.10.10
REMOTE_NET_PUBLIC=10.10.12.0/30
```

ServerB (`ServerB.env`)

```
None
# Private LAN B
PUBLIC_NET_SUBNET=192.168.20.0/24
HOST_IP=192.168.20.10
REMOTE_NET_PUBLIC=10.10.12.0/30
```

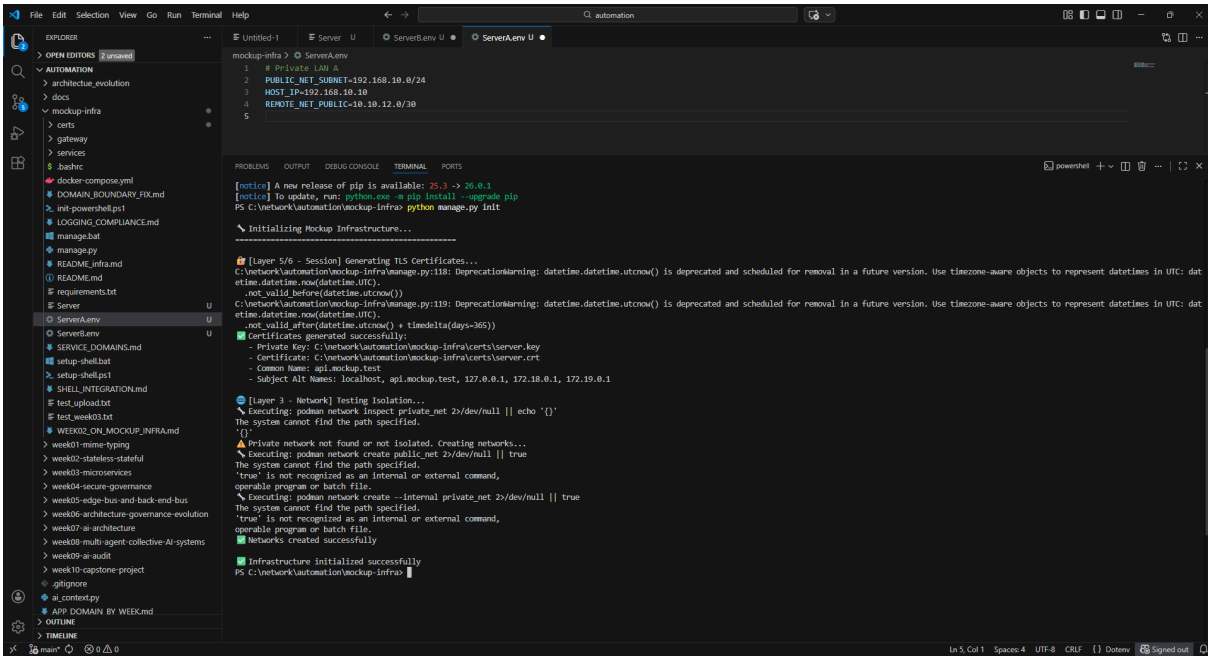
🚀 DEPLOY MOCKUP-INFRA

PowerShell (Windows Admin)

```
None
cd mockup-infra
python manage.py init
python manage.py deploy --profile week02-stateless-stateful
```

Bash (Linux)

```
Shell
cd mockup-infra
chmod +x scripts/*.sh
./scripts/init-env.sh
python manage.py deploy --profile week02-stateless-stateful
```



1. สร้างไฟล์ ENV แยกตาม Server

`ServerA.env` — ฝั่ง LAN A

env

```
None
PUBLIC_NET_SUBNET=192.168.10.0/24
HOST_IP=192.168.10.10
```

```
REMOTE_NET_PUBLIC=10.10.12.0/30
```

`ServerB.env` — ฝั่ง LAN B

env

```
None
PUBLIC_NET_SUBNET=192.168.20.0/24
HOST_IP=192.168.20.10
REMOTE_NET_PUBLIC=10.10.12.0/30
```

ไฟล์เหล่านี้กำหนดค่า IP ที่จำลองตรงกับ topology ของ R1/R2 ที่ตั้งค่าไว้

---

2. Deploy Mockup Infrastructure

Windows (PowerShell)

powershell

```
None
cd mockup-infra
python manage.py init
python manage.py deploy --profile week02-stateless-stateful
```

🟡 VERIFY CONTAINERS

```
None
podman ps
```

Should show:

Container	Port
stateless-api	3000
stateful-api	3001

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                                                                    powershell

Getting image source signatures
Copying blob sha256:955a8478f9aceb66cbf2f579fa3c24e1af278d1fa3ffd3043d6260e21d2f7416
Copying blob sha256:bca5d04786e112d958f100a66f8257b2aeefc14b64d81e405c3c44acff2fb000
Copying blob sha256:3e2c181db1b0985ce357c7aaf48ac615f30f392cd15d5c5ba34c4faa1f4f39a2
Copying blob sha256:589002ba0eaed121a1dbf42f6648f29e5be55d5c8a6ee0f8eaa0285cc21ac153
Copying blob sha256:6b7b6c7061b76cdb8601e18722d12ae3232f0ddcfa1d2983754abcc6ce0a8a83
Copying blob sha256:399d0898a94e0084f81499a3e3c29824357118c7ce551648ea1dbab813884661
Copying blob sha256:6d397a54a185dd0b6638d1a3934b81daef7a140741e12697377d6279066f7ca7
Copying blob sha256:5e7756927bef33a266e1221356d5da8553139cb80bc5b1b3827010811d9ea268
Copying config sha256:b76de378d57272a1dd9091a05dd548a3639dfb792ebdbf95d06704d2950afdea
Writing manifest to image destination
229d5578e0092d629d0bdc4830a38b2c96f25e635a8aaa723dd3333bd95aa9ab
mockup-public-web
mockup-intranet-api
mockup-gateway
PS C:\network\automation\mockup-infra> podman ps
CONTAINER ID  IMAGE                                  COMMAND               CREATED        STATUS         PORTS                                   NAMES
11ae78d2f83e  localhost/mockup-infra_public_app:latest  python app.py         52 seconds ago  Up 43 seconds  80/tcp                                  mockup-public-web
88a52a35a4be  localhost/mockup-infra_intranet_api:latest  python api.py         51 seconds ago  Up 43 seconds  5000/tcp                                mockup-intranet-api
229d5578e009  docker.io/library/nginx:alpine         nginx -g daemon o...  43 seconds ago  Up 42 seconds  0.0.0.0:443->443/tcp, 0.0.0.0:8080->80/tcp  mockup-gateway
PS C:\network\automation\mockup-infra>
```

ภาพนี้แสดงขั้นตอนการใช้งาน mockup-infra ใน VS Code เพื่อตั้งค่าและเตรียมระบบเซิร์ฟเวอร์จำลอง (ServerA) โดยสรุปสิ่งที่เกิดขึ้นในหน้าจอได้ดังนี้

1. การตั้งค่า Environment (.env)

ในส่วนบนของภาพมีการแก้ไขไฟล์ `ServerA.env` เพื่อกำหนดค่าเครือข่าย ได้แก่ `PUBLIC_NET_SUBNET=192.168.10.0/24` สำหรับวงเครือข่ายภายใน (Private LAN A), `HOST_IP=192.168.10.10` สำหรับ IP ของ ServerA และ `REMOTE_NET_PUBLIC=10.10.12.0/30` สำหรับวงเครือข่ายที่ใช้เชื่อมต่อไปยัง Site อื่น

2. การ Initialize ระบบใน Terminal

ในส่วน Terminal มีการรันคำสั่ง `python manage.py init` ซึ่งระบบดำเนินการ 3 ขั้นตอน คือ สร้าง TLS Certificate สำหรับการเชื่อมต่อแบบ HTTPS, สร้าง Network เสมือนเพื่อจำลองการแยกวง (Private vs Public) และแสดงข้อความ Infrastructure initialized successfully ยืนยันว่าระบบพร้อมสำหรับขั้นตอน Deploy ต่อไป

โดยรวมแล้วนี่คือการใช้แนวคิด Infrastructure as Code (IaC) เพื่อสร้างสภาพแวดล้อมจำลองตามที่กำหนดในไฟล์ `.env` ก่อนที่จะรัน Stateless และ Stateful API จริง

```
PROBLEMS    OUTPUT    TERMINAL    ···                    powershell + ⌄ ···

teway
PS C:\network\automation\mockup-infra> curl http://localhost:8080


StatusCode        : 200
StatusDescription : OK
Content           : <!DOCTYPE html>
                    <html lang="en">
                    <head>
                        <meta charset="UTF-8">
                        <title>Public Internet Server</title>
                        <style>
                            body {
                                font-family: -apple-system, BlinkMacSystemFont,
                    'Seg...
RawContent        : HTTP/1.1 200 OK
                    Transfer-Encoding: chunked
                    Connection: keep-alive
                    Content-Type: text/html
                    Date: Wed, 04 Mar 2026 12:12:01 GMT
                    Server: nginx/1.29.5

                    <!DOCTYPE html>
                    <html lang="en">
                    <head>
                        <...
Forms             : {}
Headers           : {[Transfer-Encoding, chunked], [Connection, keep-alive],
                    [Content-Type, text/html], [Date, Wed, 04 Mar 2026 12:12:01
                    GMT]...}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : mshtml.HTMLDocumentClass
RawContentLength  : 1323
```

```
PS C:\network\automation\mockup-infra> podman exec mockup-gateway curl http://mockup-intranet-api:5000/info
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0     0     0  --:--:-- --:--:-- --:--:--     0{"error":"Endpoint not found"}
100    31  100    31    0     0  9841     0  --:--:-- --:--:-- --:--:-- 15500
PS C:\network\automation\mockup-infra>
```

3. ผลการทดสอบการเชื่อมต่อ

ระบบตอบกลับเป็น JSON ซึ่งยืนยันว่า Gateway สามารถเรียก Intranet API ภายในวงแลนจำลองได้สำเร็จตามที่ออกแบบไว้ใน Layer 3

4. การพิสูจน์ระบบ Isolation

เมื่อทดสอบจากฝั่ง Windows (ภายนอก) ระบบแสดง Unable to connect แต่เมื่อทดสอบจากภายใน Gateway กลับตอบสนองได้ปกติ นี่คือหลักฐานที่ชัดเจนว่าระบบทำงานตามหลัก "คนนอกเข้าไม่ได้ แต่คนในสื่อสารกันได้" ตรงตามโจทย์ที่ตั้งไว้

5. Endpoint not found

ข้อความนี้ไม่ใช่ข้อผิดพลาดด้านเครือข่าย แต่เป็นเพียงการที่ยังไม่ได้ระบุ Path ที่ถูกต้องในโค้ด ในแง่ของ Network ถือว่าผ่านแล้ว

🧪 PART 3 — TESTING STATELESS vs STATEFUL (week03-stateless-stateful/phase1-mockup/README.md)

ขั้นตอนการเริ่มระบบ Infrastructure as Code (IaC) ใน VS Code เพื่อเตรียม Server จำลอง มี 3 ส่วนหลักดังนี้

1. การตั้งค่า Environment

แก้ไขไฟล์ `.env` เช่น `ServerA.env` เพื่อกำหนดค่าเครือข่ายพื้นฐาน ได้แก่ วง LAN (`192.168.10.0/24`) และ IP ของ Server (`192.168.10.10`)

2. การรันคำสั่ง

พิมพ์คำสั่ง `python manage.py init` ใน Terminal เพื่อเริ่มกระบวนการเตรียมระบบ

3. สิ่งที่ระบบทำอัตโนมัติ

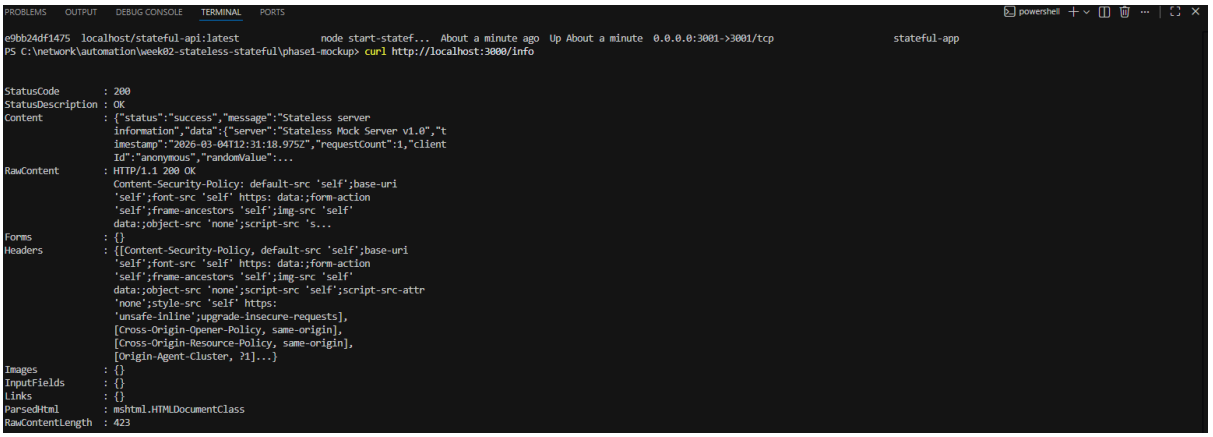
ระบบดำเนินการ 3 ขั้นตอนต่อเนื่องกัน คือ สร้าง TLS Certificate สำหรับการเชื่อมต่อ HTTPS, สร้างเครือข่ายจำลองแบบแยกส่วน (Private/Public) ใน Docker/Podman และแสดงข้อความ `Infrastructure initialized successfully` เพื่อยืนยันว่าระบบพร้อมสำหรับการ Deploy บริการ API ในขั้นตอนถัดไป

⚫ LOCAL TESTS (Private LAN)

From ClientA:

```shell
curl http://192.168.10.10:3000/info
```

→ Stateless



ขั้นตอนการ Deploy หลังจาก Init ระบบเสร็จแล้ว มีรายละเอียดดังนี้

คำสั่งที่ใช้

bash

```shell
python manage.py deploy --profile week02-stateless-stateful
```

สิ่งที่เกิดขึ้นใน Terminal

ระบบทำงานต่อเนื่อง 3 ขั้นตอน คือ ดึงค่า Config ของโปรเจกต์ `week02-stateless-stateful` มาใช้งาน (Loading Profile), สร้างและรัน Container สำหรับ API ทั้งสองรูปแบบ (Starting Container Group) และเปิดให้บริการ `stateless-api` ที่พอร์ต 3000 สำหรับการทำงานแบบไม่เก็บ session และ `stateful-api` ที่พอร์ต 3001 สำหรับการทำงานแบบจดจำ session ของผู้ใช้

สถานะสุดท้าย

ข้อความ `Deployment successful!` ยืนยันว่า Server จำลองพร้อมให้บริการและรอรับการเชื่อมต่อจาก Client ผ่าน Router ที่ตั้งค่าไว้แล้ว

From ClientA:

```shell
curl -X POST http://192.168.10.10:3001/session
```

Store the returned `Session-ID`, then:

```shell
curl http://192.168.10.10:3001/data -H "Session-ID: <id>"
```

🔵 REMOTE TESTS (Via Simulated Internet)

From ClientA:

```
curl http://10.10.12.2:3000/info
```

That will target **ServerB stateless** — NAT + routing should deliver

---

Session Test:

```
curl -X POST http://10.10.12.2:3001/session
```

→ ServerB returns a session ID

Then:

```
None
curl http://10.10.12.2:3001/data -H "Session-ID: <id>"
```

Check session persists across NAT

⚠️ TROUBLESHOOTING

❗ Cannot Reach Remote

- Gateway misconfiguration
- NAT ACL incorrect
- Wrong default gateway on clients

Check:

```
None
show ip route
ping 10.10.12.2 from R1
```

❗ Sessions Lose State

- Missing header
- Wrong container port
- Firewall blocking port 3001

🧠 ENGINEERING REFLECTION

1. **Stateless services don't depend on prior data**
2. **Stateful services require session headers**
3. NAT shouldn't affect logical session identity

📄 LAB README.md (for repository)

```
None
# Lab 4 — Stateful vs Stateless Across Public & Private Subnets

## Architecture
Two LANs connected via a simulated internet (10.10.12.0/30)
- LAN A: 192.168.10.0/24
- LAN B: 192.168.20.0/24
Routers provide static routing + NAT.

## Steps

### 1. Setup Network
Configure R1 and R2:
- Static routes
- NAT overload
- Verify ping

### 2. Deploy IaC
Configure `ServerA.env` and `ServerB.env`
Deploy via VS Code, PowerShell or Bash using:
```

python manage.py deploy --profile week02-stateless-stateful

```
None

### 3. Validate
- Local stateless vs stateful
- Remote stateless vs stateful
- Test session persistence
- Capture NAT translations

## Troubleshooting
- Check interface status, NAT ACLs
- Verify client gateways

## Extensions
- OSPF instead of static routing
- Load balancer in front of APIs
```